

گزارش نهایی پروژه – نوا (Nava)

سرویس استریم موسیقی – درس برنامه‌سازی وب، بهار ۱۴۰۵، دانشگاه صنعتی شریف

۱. اعضای تیم و تقسیم کار

عضو	ایمیل	نقش کلی
نیما علی‌قارداشی	nimaalighardashi@gmail.com	معماری، احراز هویت، تحلیل‌ها/گزارش‌ها، داکر
امیرحسین اباذری	amirhossein.abazari@yahoo.com	اشتراک و پرداخت، تنظیمات، اعلان‌ها، تیکت‌ها
آروین طاهری	arvintaheri100@gmail.com	کاتالوگ و استودیو، پخش‌کننده، آزمون‌ها

تقسیم کار به تفکیک فاز

فاز ۱ (فرانت‌اند، Next.js): نیما اسکلت پروژه، احراز هویت و مسیرها؛ آروین کاتالوگ، کارت‌های موسیقی و پخش‌کننده؛ امیرحسین صفحات اشتراک، اعلان‌ها و پشتیبانی را ساختند.

فاز ۲ (بک‌اند + یکپارچه‌سازی): همان تقسیم‌بندی حفظ شد تا هر کس روی حوزه‌ای که در فاز ۱ ساخته بود، سمت سرور را هم پیاده کند.

۲. قراردادهای توسعه

- **Conventional Commits** با پیشوندهای `feat / fix / chore / test / docs` و `scope` (مثل `feat(backend/billing): ...`).
- **دروازه‌های کیفیت** پیش از هر `merge`: در فرانت‌اند `pnpm lint`، `pnpm typecheck`، `pnpm test` (۵۰ آزمون) و `pnpm build`؛ در بک‌اند `uv run pytest` (۳۸ آزمون) و `manage.py check`.
- **قرارداد API** با **camelCase**: مدل‌های جنگو `snake_case` هستند اما با `django-rest-framework` خروجی/ورودی JSON به صورت `camelCase` تبدیل می‌شود؛ در نتیجه `lib/types.ts` تنها منبع حقیقت انواع باقی می‌ماند و هیچ فایل فرانتی برای فاز ۲ تغییر نکرد.
- **هم‌مبدأ بودن (same-origin)**: فرانت‌اند فقط با `/api` و `/media` صحبت می‌کند و `next.config.mjs` این‌ها را به بک‌اند (`localhost:8321` یا `backend:8321` در داکر) پروکسی می‌کند؛ بنابراین کوکی نشست و توکن CSRF بدون CORS کار می‌کنند.

۳. ساختار پروژه

فرانت‌اند (فاز ۱، بازاستفاده‌شده در فاز ۲)

```
app/           # خانه، مرور، استودیو، تنظیمات، داشبورد: مسیرهای Next.js (App Router)
components/    # بوت‌استرپ + همگام‌سازی تنظیمات + پخش‌کننده + UI کامپوننت‌های
lib/           # HTTP به LocalStorage امضای توابع ثابت، بدنه از: مخازن (repositories)
  api/         # کوچید
queries/       # React Query هوک‌های
types.ts       # (User, Track, ...) منبع حقیقت انواع
subscriptions.ts # جدول سطوح اشتراک (نسخه سمت کلاینت)
store/         # (Zustand) وضعیت احراز هویت و پخش‌کننده
```

بک‌اند (فاز ۲، Django + DRF)

```
backend/
  config/      # settings (env مبتنی بر)، urls، wsgi
  core/        # short_id (کلید اولیه رشته‌ای)، tiers.py، permissions، exceptions
  accounts/    # User سفارشی، Artist، Verification، follows، UserSettings، auth
  catalog/     # Album، Track، مرور، دانلود، دسترسی زودهنگام، آپلود چندبخشی،
  engagement/  # Playlist، StreamEvent، RecentItem، Notification، سرویس‌ها
  billing/     # SubscriptionPlan، Payment، gateways/ (الگوی Strategy)
  analytics/   # recommendations.py، حسابداری ماهانه، تجمیع‌ها
```

۴. مدل‌های داده و منطق روابط

- **accounts.User** (وارث `AbstractUser`، ورود با ایمیل): نقش (listener/artist/support/admin) و سطح (basic/silver/gold) و `subscription_expires_at`. خصیصه محاسبه‌شده `effective_tier` اگر اشتراک منقضی شده باشد `basic` برمی‌گرداند؛ در نتیجه انقضا بدون `cron` و در همه نقاطی که یک گیت آن را می‌خواند اعمال می‌شود.
- **روابط دنبال‌کردن** (`UserFollow`، `ArtistFollow`): یال‌های گراف اجتماعی با `unique_together`. شمارنده‌های نمایشی (`denormalize`) `follower_count`، `monthly_listeners` شده‌اند تا اعداد دمو‌ی فاز ۱ و تغییرات زنده کاربر کنار هم بمانند.
- **catalog.Track**: فایل صوتی واقعی (`FileField`) با اعتبارسنجی پسوند (mp3/wav/flac) به‌همراه `source_url` به‌عنوان جایگزین (برای صوت‌های دمو بسته‌بندی‌شده). `duration` هنگام آپلود با `mutagen` استخراج می‌شود. پرچم `early_access` مبنای گیت سطح `gold` است.

- **engagement.Playlist + PlaylistTrack**: مدل واسط با فیلد `position` ترتیب آهنگ‌ها را حفظ می‌کند.
 - **billing.SubscriptionPlan**: دو ردیف (silver/gold) با `monthly_price` که ادمین در زمان اجرا آن را عوض می‌کند (بدون تغییر کد). **Payment** مبلغ را سمت سرور به صورت `months × price` محاسبه می‌کند.
 - **analytics.ArtistMonthlyAccount**: تسویه ماهانه هنرمند؛ ردیف ماه جاری هنگام خواندن از روی `StreamEvent` بازمحاسبه می‌شود و ردیف‌های تسویه شده منجمد می‌مانند.
- کلید اولیه همه موجودیت‌های اصلی رشته‌ای است (`core.models.short_id`) تا شناسه‌های فاز ۱ (`u_admin`, `ar_navid`, `tr_...`) عیناً بازاستفاده شوند و لینک‌های عمیق دمو دقیقاً مثل قبل کار کنند.

۵. توجه نگه‌داری‌پذیری (Maintainability)

1. درز مخزن (**repository seam**): کامپوننت‌ها → هوک‌های `lib/api/*` → React Query → بک‌اند. چون امضای ~۵۵ تابع مخزن ثابت ماند، کل کوچ فاز ۲ فقط تعویض بدنه همین توابع بود و هیچ کامپوننتی دست نخورد.
2. یک منبع حقیقت برای سطوح اشتراک: `core/tiers.py` آینه دقیق `lib/subscriptions.ts` است؛ هر محدودیت (۶۰ استریم/روز، سقف پلی‌لیست، پرچم‌های دانلود/آواتار/دسترسی‌زودهنگام/آمار) سمت سرور پیش از هر نوشتن دوباره بررسی می‌شود (مرورگر مورد اعتماد نیست).
3. لایه سرویس: منطق دامنه (ثبت استریم، انتشار اثر، اعمال اشتراک، تجمیع‌ها) در `services.py` هر اپ جدا از ویوها نگه‌داری می‌شود و مستقیماً واحدآزمون‌پذیر است.
4. الگوی Strategy برای درگاه پرداخت: `PaymentGateway` (ABC) با دو پیاده‌سازی `MockGateway` و `ZarinpalGateway`؛ انتخاب با `get_gateway()` از روی `settings.PAYMENT_GATEWAY`. افزودن درگاه جدید یعنی یک کلاس تازه، بدون دست‌زدن به ویوها.

۶. قابلیت‌های کلیدی سمت سرور

- کنترل دسترسی نقش/سطح: کلاس‌های `permission` در `core/permissions.py` و گیت‌های سطح که همگی `effective_tier` را می‌خوانند. خطاهای تجاری با کد ماشین‌خوان برمی‌گردند: ۴۲۹ با `stream_limit` و ۴۰۳ با `playlist_limit` که فرانت‌اند روی آن‌ها شاخه می‌زند.
- آپلود/دانلود فایل: انتشار اثر به صورت `multipart` (صوت + کاور)، دانلود گیت‌شده با `FileResponse`.
- پرداخت با قیمت زمان‌اجرا: خرید ۱۲/۶/۳/۱ ماهه، بازگشت از درگاه، تمدید اشتراک روی `max(now, ...)` (انقضای فعلی).

- همگام‌سازی تنظیمات (§۳.۵): زبان و پوسته کاربر در `UserSettings` ذخیره و بین دستگاه‌ها همگام می‌شود.
- تجميع سمت سرور: توزیع کاربران و درآمد ماهانه صرفاً با aggregate های ORM محاسبه می‌شوند؛ فرانت هرگز فهرست خام کاربران را نمی‌گیرد.
- سیستم پیشنهاد (امتیازی): `analytics/recommendations.py` بر پایه تاریخچه ۳۰ روز اخیر امتیازدهی می‌کند: هنرمند دنبال‌شده + ۲×وزن ژانر + ۱×وزن هنرمند + گرۀ کوچک محبوبیت×۳؛ آهنگ‌های شنیده‌شده و دسترسی‌زود هنگام غیرمجاز حذف می‌شوند. قطعی و قابل توضیح است، نه تصادفی.

۷. نقش هوش مصنوعی

از دستیار هوش مصنوعی (Claude Code) به عنوان جفت برنامه نویس استفاده شد: طراحی معماری بک‌اند مطابق قرارداد موجود فرانت‌اند، تولید اسکلت اپ‌ها و مدل‌ها، و نوشتن آزمون‌ها.

نمونه کد تولیدشده با کمک AI (فاز ۱): لایه LocalStorage تایپ‌دار در `lib/db/storage.ts`. این فایل در فاز ۲ طبق طراحی حذف شد (تعویض بدنه مخزن‌ها با HTTP)؛ متن کامل آن در تاریخچه گیت موجود است:

```
// lib/db/storage.ts (فاز ۱ - در فاز ۲ حذف شد)
const PREFIX = "nava:"

export function read<T>(key: string, fallback: T): T {
  if (!isBrowser()) return fallback
  try {
    const raw = window.localStorage.getItem(PREFIX + key)
    return raw === null ? fallback : (JSON.parse(raw) as T)
  } catch {
    return fallback
  }
}

export function write<T>(key: string, value: T): void {
  if (!isBrowser()) return
  window.localStorage.setItem(PREFIX + key, JSON.stringify(value))
}
```

نمونه کد تولیدشده با کمک AI (فاز ۲): موتور پیشنهاد در `backend/analytics/recommendations.py` — امتیازدهی وزنی بر پایه ژانر/هنرمند از رویدادهای استریم ۳۰ روز اخیر، به همراه fallback محبوبیت برای کاربران تازه:

```
# backend/analytics/recommendations.py - هسته امتیازدهی
followed = set(user.following_artists.values_list("artist_id", flat=True))

candidates = visible_tracks(user).exclude(id__in=heard_ids)
```

```

scored = []
for track in candidates:
    score = 0.0
    if track.artist_id in followed:
        score += 3
    score += 2 * genre_weight.get(track.genre, 0)
    score += 1 * artist_weight.get(track.artist_id, 0)
    score += min(track.streams, 100000) / 1_000_000 # tiny popularity nudge
    if score > 0:
        scored.append((score, track))

# fallback کاربر بدون تاریخچه: پرشونده‌ترین‌ها به‌عنوان
if not scored:
    return list(candidates.order_by("-streams")[:limit])

```

نقاط قوت AI: سرعت در کارهای تکراری (تعویض بدنهٔ ۱۰ مخزن با امضای یکسان)، رعایت قراردادها (camelCase، CSRF، slashless)، و تولید آزمون‌های جامع.

نقاط ضعف AI: گاهی جزئیات محیط را اشتباه می‌گرفت (مثلاً `django.db.sqlite3` به‌جای `django.db.backends.sqlite3`، یا سریال‌سازی‌پذیر نبودن closure در مهاجرت‌ها) که نیاز به بازبینی انسانی داشت؛ و ناسازگاری ابزارها (`ky/jsdom/undici` در آزمون‌ها) که با تصمیم انسانی حل شد.

۸. اجرا

با داکر (توصیه‌شده):

```

docker compose up --build
# فرانت‌اند: http://localhost:3000

```

دستی:

```

# بک‌اند
cd backend && uv run python manage.py migrate && uv run python manage.py seed
uv run python manage.py runserver 8321
# فرانت‌اند (در ترمینال دیگر)
pnpm install && pnpm dev

```

حساب‌های دمو (رمز همه: `admin@nava.app`, `support@nava.app`, `artist@nava.app`, `listener@nava.app` (طلایی)، `silver@nava.app` (نقره‌ای)، `basic@nava.app` (پایه).

مستندات API: پس از اجرای بک‌اند در (Swagger) `http://localhost:8321/api/docs/`.